Numpy

```
# Install Numpy
!pip install numpy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
# Import Numpy library
import numpy as np
```

```
print(np.__version__) # Numpy version installed in the system
```

```
2.0.1
```

1. creating arrays

```
list1 = [1,2,3]
print(list1)
```

```
[1, 2, 3]
```

```
arr_list1 = np.array(list1) # creating array using list
print(arr_list1)
```

```
[1 2 3]
```

```
print(type(arr_list1))  # check data type of variable
```

```
<class 'numpy.ndarray'>
```

```
print(arr_list1.ndim)   # check dimension of array
```

```
1
```

```
# creating 2-dim array using 2 lists
list1 = [1,2,3]
list2 = [4,5,6]
arr_list1_2 = np.array([list1, list2])
print(arr_list1_2)
```

```
[[1 2 3]
 [4 5 6]]
```

```
print(arr_list1_2.ndim) # check dimension of array
```

```
2
```

```
# creating 3-dim array using 3 lists
list1 = [1,2,3]
list2 = [4,5,6]
list3 = [7,8,9]
arr_list1_3 = np.array([[list1, list2, list3]])
print(arr_list1_3)
```

```
[[[1 2 3]
  [4 5 6]
  [7 8 9]]]
```

```
print(arr_list1_3.ndim)     # check dimension of array
```

```
3
```

2. numpy array attributes

```
arr1 = np.array([[1,2,3], [4,5,6]])
print(arr1)
```

```
[[1 2 3]
 [4 5 6]]
```

```
# Attributes of Array (properties of array)
print("Shape", arr1.shape)  # share= rows, columns and height of array
print("Size", arr1.size)    # number of elements in array
print("dType", arr1.dtype)  # data type of elements in array
print("n_dim", arr1.ndim)   # dimension of array
```

```
Shape (2, 3)
Size 6
dType int64
n_dim 2
```

```
# HW: create two arrays - 1D and 3D, and find it's attributes
```

3. Array initialization methods

```
# zeros array : all elements are 0

zero_arr = np.zeros((2,3))
print(zero_arr)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
# ones array : all elements are 1

one_arr = np.ones((1,3))
print(one_arr)
```

```
[[1. 1. 1.]]
```

```
# Full array : all elements are same based on given value

full_arr = np.full((3,2), 9)
print(full_arr)
```

```
[[9 9]
 [9 9]
 [9 9]]
```

```
# Identity Matrix : diagonal elements are ones and all other elements are zeros

id_arr = np.eye(3)
print(id_arr)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
# empty array
print(np.empty(2))
```

```
[1.20166646e-311 0.00000000e+000]
```

```
# evenly spaced array : elements are evenly spaced based on step value
print(np.arange(2,10,2))
```

```
[2 4 6 8]
```

```
# specific number of eqaully spaced values between a range : elements are equally spaced based on step value
print(np.linspace(1,10,4))
```

```
[ 1.  4.  7. 10.]
```

```
# random values array - float
r_arr = np.random.rand(3,2)
print(r_arr)
```

```
[[0.99189518 0.36638098]
 [0.38564405 0.5403712 ]
 [0.70119322 0.29187343]]
```

```
# random values array - int
rint_arr = np.random.randint(1,100,(3,3))
print(rint_arr)
```

```
[[60 13 62]
 [ 3 36 14]
 [96 78 10]]
```

4 Array Indexing and Slicing

```
a = np.array([1,3,5,7,9])
print(a)
```

```
[1 3 5 7 9]
```

```
print(a[0])     # first element of array
```

```
1
```

```
print(a[4])     # fifth element of array
```

```
9
```

```
print(a[-1])      # last element of array
```

```
9
```

```
a = np.array([1,3,5,7,9])
print(a)
# Slicing :
# array[start:stop:step]
# stop - index, that is not included in output
```

```
[1 3 5 7 9]
```

```
print(a[0:3]) # First 3 elements
```

```
[1 3 5]
```

```
print(a[-3:]) # Last 3 elements
```

```
[5 7 9]
```

```
print(a[1:4]) # Middle 3 elements
```

```
print(a[0::3]) # Middle 3 elements with steps
```

```
print(a[0::2]) # All alternative elements with steps
```

```
# Indexing on 2 Dimensional Array
arr2 = np.array([[1,2,3], [4,5,6]])
print(arr2)

# 2D Array: [rows, columns]                  # 2D Array takes 2 values - rows and columns for indexing
# 3D Array: [layers/height, rows, columns]   # 3D Array takes 3 values - height, rows and columns for indexing
```

```
[[1 2 3]
 [4 5 6]]
```

```
print(arr2[0]) # first row
print(arr2[1]) # second row
```

```
[1 2 3]
[4 5 6]
```

```
# Slicing on 2D array
print(arr2[0][0]) # element of first row and first column
print(arr2[0][1]) # element of first row and second column
print(arr2[1][2]) # element of second row and third column
```

```
1
2
6
```

```
print(arr2[1]) # specific element - 1D
print(arr2[1:]) # slicing part - 2D
```

```
[4 5 6]
[[4 5 6]]
```

```
print(arr2)
```

```
# 2D Array: [rows, columns]  |  eg: arr2[0][1]     # 2D Array takes 2 values - rows and columns for indexing
```

```
[[1 2 3]
 [4 5 6]]
```

```
print(arr2[:, 0])       # First column of an array
print(arr2[:, 1])       # Second column of an array
print(arr2[:, 2])       # Third column of an array
```

```
[1 4]
```

```
arr3 = np.array([[1,2,3],
                 [4,5,6],
                 [7,8,9]])
print(arr3)

# 3D Array: [layers/height, rows, columns]        # 3D Array takes 3 values - height, rows and columns for indexing
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
print(arr3[0])
print(arr3[1])
print(arr3[2])
```

```
[1 2 3]
[4 5 6]
[7 8 9]
```

```
print(arr3[0][2])
print(arr3[2][2])
```

```
3
9
```

```
print(arr3[:, 0]) # 1st col value
print(arr3[:, 1]) # 2nd col value
print(arr3[:, 2]) # 3rd col value
```

```
[1 4 7]
[3 6 9]
```

```
print(arr3[::, 1:2])  #slicing on columns
print(arr3[2::, 1:2]) #slicing on rows and columns
```

```
[[2]
 [5]
 [8]]
[[8]]
```

5 Array Reshaping and Flattening

```
# Reshape Array: changing its dimensions (e.g., rows and columns) without altering the actual data
# Flatten Array: transforming a multi-dimensional array into a single-dimensional array
```

```
arr1 = np.array([1,2,3,4,5,6])      # shape = (1,6) - 1 row and 6 columns
print(arr1)
```

```
[1 2 3 4 5 6]
```

```
reshaped = arr1.reshape((6,1))      # shape = (6,1) - 6 rows and 1 column
print(reshaped)
```

```
[[1]
 [2]
 [3]
 [4]
 [5]
 [6]]
```

```
reshaped2 = arr1.reshape((2,3))     # re-shaped array from (1,6) to (2,3)
print(reshaped2)
```

```
[[1 2 3]
 [4 5 6]]
```

```
print(reshaped2.flatten())          # flattened array from higher (2D) dimensional (2,3) to (1,6) (1D)
```

```
[1 2 3 4 5 6]
```

6 Mathematical Operations on Array

```
a = np.array([10,20,40,-30])
print(a)
```

```
[ 10  20  40 -30]
```

```
print(a+10) # add a value in array
print(a-30) # subs a value in array
print(a*2) # multiply by a value in array
print(a/10) # divide by a value in array
```

```
[ 20  30  50 -20]
[-20 -10  10 -60]
[ 20  40  80 -60]
[ 1.  2.  4. -3.]
```

```
b = np.array([1,4,9])
print(b)
```

```
[1 4 9]
```

```
print(np.square(b)) # square of all array elements
print(np.sqrt(b))   # square root of all array elements
```

```
[ 1 16 81]
[1. 2. 3.]
```

```
# HW: sin, tan, cos, sec
```

7 Mathematical Operations on Muliple Arrays

```
a = np.array([1,2,3])
b = np.array([4,5,6])
print(a)
print(b)
```

```
[1 2 3]
[4 5 6]
```

```
print(np.add(a,b))          # add two arrays
print(np.subtract(a,b))     # subtract two arrays
print(np.multiply(a,b))     # multiply two arrays
print(np.divide(a,b))       # divide two arrays
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
```

```
# Dot product: computes the inner product of the two vectors and sum the results
print(np.dot(a,b)) # dot product
```

```
32
```

```
print(a.T) # Transpose: Returns an array with axes transposed
```

```
[1 2 3]
```

```
t = np.array([[1,2,3],
              [4,5,6]])

print(t)    # initial shape: 2,3
print(t.T)  # shape after transpose: 3,2
```

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

8 Statistical Functions

```
a = np.array([[1,2,3], [4,5,6]])
print(a)
```

```
[[1 2 3]
 [4 5 6]]
```

```
print(np.sum(a))    # sum of all elements of an array
```

```
21
```

```
print(np.mean(a))
print(np.median(a))
print(np.std(a))
print(np.min(a))
print(np.max(a))
```

```
3.5
3.5
1.707825127659933
1
6
```

```
# This is formatted as code
```

9 Array Comparison

```
a = np.array([1,5,3])
b = np.array([4,5,6])
print(a)
print(b)
```

```
[1 5 3]
[4 5 6]
```

```
print(a == b) # comparing each elements of an array
print(np.array_equal(a,b)) # comparing complete array
```

```
[False  True False]
False
```

10 Broadcasting

```
# Broadcasting: is a mechanism that allows arithmetic operations to be performed on arrays of different shapes and sizes,
# it eliminates the need for explicit loops or reshaping operations, making code more concise, readable and efficient.

# Broadcasting Rules:
    # Compare shapes from right to left.
    # Dimensions must match or be 1.
    # If one array has fewer dimensions, pad its shape on the left with 1s.
    # Any dimension equal to 1 can stretch to match the other array.
```

```
a = np.array([1,2,3,4])
b = np.array([5])
print(a+b)      # adding 1D with 0ne-element array to 1D with multi-element array
```

```
[6 7 8 9]
```

```
c = np.array([[1,2,3], [4,5,6]])
d = np.array([10,20,30])
print(c+d)      # adding 2D with 6-element array (2,3) to 1D with 6-element array
```

```
[[11 22 33]
 [14 25 36]]
```

```
# Homework:
# 1. Create 5X5 matrix with values from 1 to 25
  # > extract last row
  # > first column
  # > sub matrix containing 3X3

# 2. Broadcasting: multiply shape- (3,1) matrix with shape- (3,)
```